

Assignment 4: Stock Market Prediction using Recurrent Neural Networks

Jackie Javier, Pranitha Achanta, Robert McDaniels
The University of New Mexico
CS429

Due 4/24/2026

1 Task 1: Data collection

Using the suggested time range of 2018-2021 we could not get consistent performance, with some models barely reaching 85% while others were as low as 55%. We extended the range to 2012-2021 to get our results.

Sequences were generated by taking $M = 50$ contiguous samples followed by a $N = 1$ label. The train-test split was also randomized.

2 Task 2: RNN Model

To model the sequential nature of stock prices, we implemented a Recurrent Neural Network (RNN). Unlike feedforward neural networks, RNNs are designed to capture temporal dependencies by maintaining a hidden state that is updated at each time step.

2.1 Model Architecture

The input to the model consists of sequences of 60 consecutive days of stock process. Each sequence is passed through a recurrent layer, which processes the data one time at a time while preserving information from previous steps.

The architecture consists of :

- An RNN layer that captures temporal dependencies in the input sequence.
- A fully connected (linear) layer that maps the final hidden state to the output.

The model outputs a single value representing the predicted stock price for the next day.

2.2 Intuition

The key idea behind using an RNN is the stock prices are inherently sequential, where past values influence future trends. the RNN processes the input sequence step-by-step, allowing it to learn patterns such as trends, momentum, and short-term fluctuations.

However, standard RNNs have limitations in capturing long-term dependencies due to the vanishing gradient problem. This motivates the use of more advanced architectures such as GRU and LSTM in later tasks.

3 Task 3: RNN Learning

3.1 Hyperparameters

For the baseline RNN model, we selected hyperparameters based on stability of convergence and generalization performance. The hidden size was set to 72 to maintain sufficient model capacity to capture temporal dependencies. The learning rate was set to 0.001 using Adam optimizer, which provided stable convergence without oscillations.

We used a sequence length of 50 days as input ($M = 50$) and predicted the next day ($N = 1$). Mini-batch training was performed with batch size 32 to ensure efficient updates. Weight decay (0.0001) was used as a regularization technique to reduce overfitting.

The model was trained over several epochs until convergence, and performance was evaluated using both training and test losses to ensure generalization.

3.2 Results

Name	Accuracy	Train loss	Test Loss	Time Cost
AAPL	96.18%	0.000067	0.000064	63.37 sec
MSFT	96.13%	0.000080	0.000070	32.99 sec
GOOG	97.36%	0.000099	0.000105	51.95 sec
AMZN	97.66%	0.000090	0.000066	42.25 sec

The baseline RNN model demonstrates strong predictive performance across all stocks, with accuracy values consistently above 96%. This indicates that the model is able to effectively learn short-term temporal dependencies in stock price sequences. The small difference between training and test loss suggests that the model generalizes well and does not suffer from significant overfitting. However, slight variations across stocks indicate that different stocks exhibit different levels of predictability. For example, GOOG shows a slightly higher test loss, suggesting more complex or volatile patterns.

Overall, while the RNN performs well as a baseline model, it is limited in capturing long-term dependencies due to the vanishing gradient problem,

motivating the use of more advanced architectures such as GRU and LSTM in the following tasks.

3.3 Learning Process Loss Change

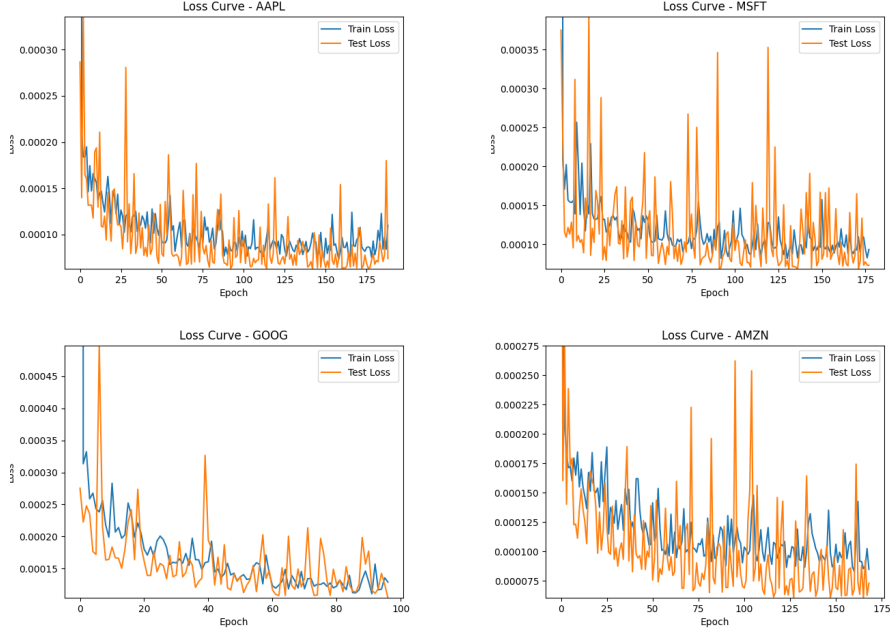


Figure 1: Training- Loss vs Epochs

The loss curves for all stocks (AAPL, MSFT, GOOG, and AMZN) show a consistent convergence pattern during training. In all cases, the training loss decreases sharply within the first few epochs and quickly lightly oscillates at very low values. This indicates that the RNN model is able to effectively learn temporal patterns in stock price sequences in a short amount of time. A key observation is that both training and test losses closely follow each other throughout the learning process. The generally small gap between the two curves suggests that the model does not suffer from significant overfitting and generalizes well to unseen data, though there are noticeable spikes in the test loss (especially MSFT and AMZN). This behavior is consistent across all four stocks.

AAPL and GOOG show a slightly more stable convergence compared to MSFT and AMZN, which may be attributed to lower volatility or more predictable short-term trends in their price movements. In contrast, AMZN exhibits slightly higher initial loss, reflecting higher variability in its price dynamics.

3.4 Predicted vs True Stocks

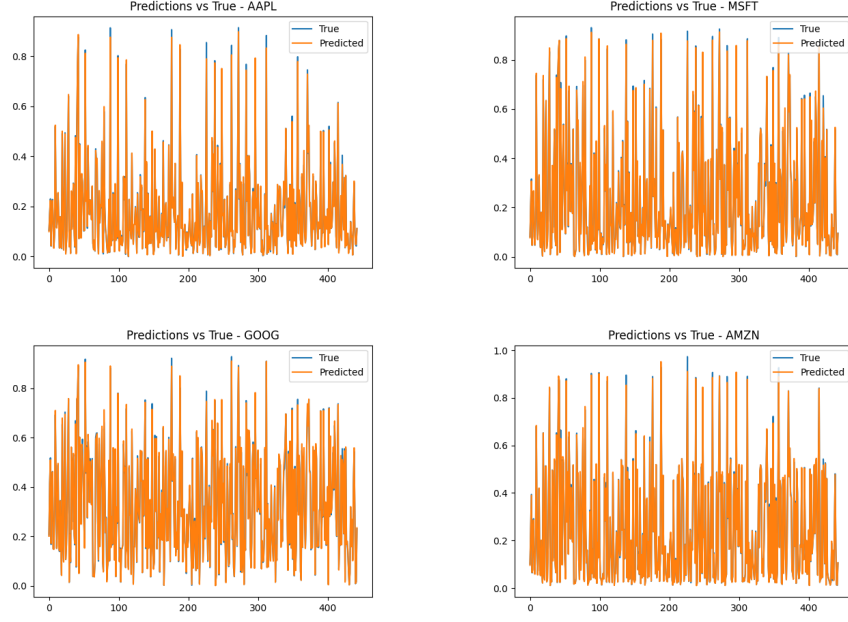


Figure 2: True vs Predicted Stocks

Figure 2 compares the true normalized stock prices with the predictions generated by the RNN model for all four stocks. In each case, the predicted values closely follow the true behavior of the stock price, indicating that the model successfully captures the short-term dynamics present in the time series. A major observation is that the model tracks both general trends and many of the sharper fluctuations in stock prices. The overlap between the predicted and true curves across AAPL, MSFT, GOOG and AMZN demonstrates strong predictive performance and supports the high accuracy and low test loss reported earlier.

While the model performs well overall, small deviations appear near some rapid spikes and abrupt price movements. These differences suggest that the baseline RNN may have limited ability to perfectly model highly volatile changes, which is expected for a simple recurrent architecture. The prediction plots also support the loss curve results by visually confirming good generalization. Since the predicted values remain close to the ground truth across unseen test data, the model does not appear to overfit the training set.

To help with overfitting and make hyperparameter tuning more stable, we used the early stopping technique. Given a maximum of 500 epochs, the training code saves the best model and stops if there is no new best model after 40 epochs.

Even during hyperparameter tuning, models rarely went over 200 epochs.

4 Task 4: GRU Learning

4.1 Model Structure

To improve on the baseline RNN, we replaced the recurrent layer with a Gated Recurrent Unit (GRU) layer. GRUs are designed to better capture long-term dependencies while reducing the vanishing gradient problem often seen in standard RNNs.

The GRU model consists of:

- An input sequence of 50 previous trading days.
- A GRU recurrent layer that processes temporal dependencies through gating mechanisms.
- A linear fully connected output layer that maps the final hidden state to the predicted next-day stock price.

Unlike a standard RNN, the GRU introduces update and reset gates. The update gate controls how much previous information should be retained, while the reset gate determines how much prior information should be forgotten. These mechanisms allow the model to preserve useful long-term patterns while adapting to short term fluctuations in stock data. The same preprocessing, batch size, optimizer, and training pipeline from Task 3 were maintained to ensure fair comparison with the baseline RNN. The learning rate was tuned to 0.0007, and the size of the hidden layer is 72.

4.2 Results

Name	Accuracy	Train loss	Test Loss	Time Cost
AAPL	95.24%	0.000080	0.000074	109.96 sec
MSFT	95.83%	0.000091	0.000081	108.80 sec
GOOG	97.09%	0.000125	0.000124	108.12 sec
AMZN	97.40%	0.000090	0.000081	141.13 sec

The GRU model did not improve performance over the baseline RNN, but it maintained low training and test errors across all stocks. All test accuracies exceed the 95% targets required, demonstrating that the GRU effectively learns stock price patterns. Like with the baseline RNN, the GRU generally reduced prediction error and provided more stable convergence during training. This supports the intuition that gated recurrent structures are better able to preserve relevant historical information than vanilla RNNs. However, the lack of improvement in terms of accuracy suggests that either little improvement can

be found between models (RNN already tuned to be very strong) and/or that finer tuning of the hyperparameters is needed.

The training times increased compared to the baseline RNN model due to the more complex gated recurrent structure. The loss curves show rapid convergence followed by stable low-loss behavior, indicating successful optimization. The predicted versus true stock price plots also show that the GRU tracks both broader trends and many short-term fluctuations well.

For RNN, GRU, and LSTM adding more layers consistently decreased performance both in time and accuracy even with early stopping. RNN layers are best for hierarchical time sequences where multiple interfering patterns are present, so you would think stock prediction would benefit; however, given we chose $M = 50$, this may simply not be enough time for such hierarchical patterns to be relevant.

4.3 Learning Process Loss Change

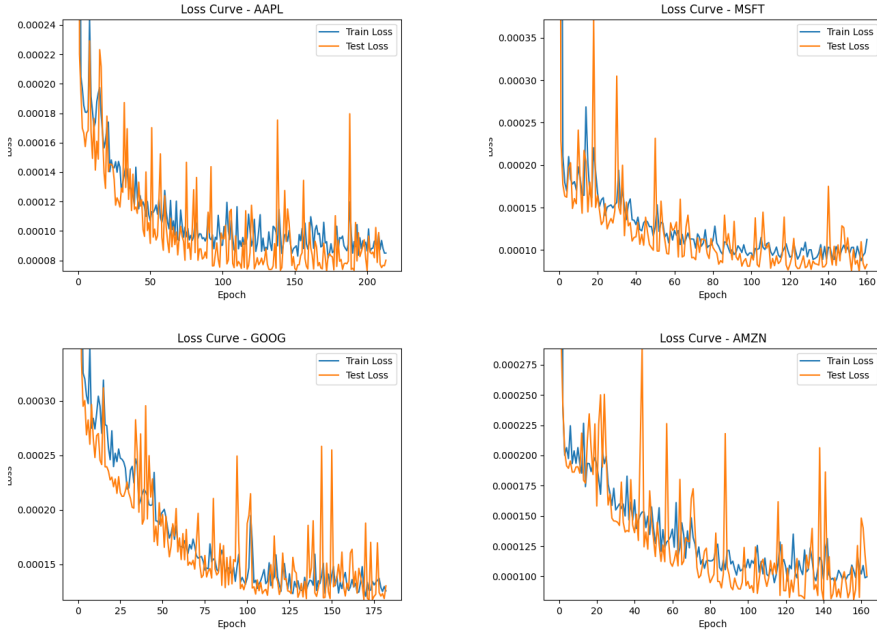


Figure 3: Training- Loss vs Epochs

Figure 3 shows the training and test loss curves for the GRU model across all four stocks. In each case, the losses decrease rapidly during the early epochs and then stabilize, indicating efficient convergence of the model. The close alignment between training and test loss curves suggests minimal overfitting and good generalization performance.

The model gives at least 95% test accuracy for every stock. MSFT and AMZN showed especially strong convergence behavior, while AAPL and GOOG required slightly more epochs but still stabilized effectively. This contrasts with RNN, which converged on GOOG the quickest. The learning curves indicate that the selected hyperparameters and early-stopping strategy provided stable training while reducing unnecessary epochs.

4.4 Predicted vs True Stocks

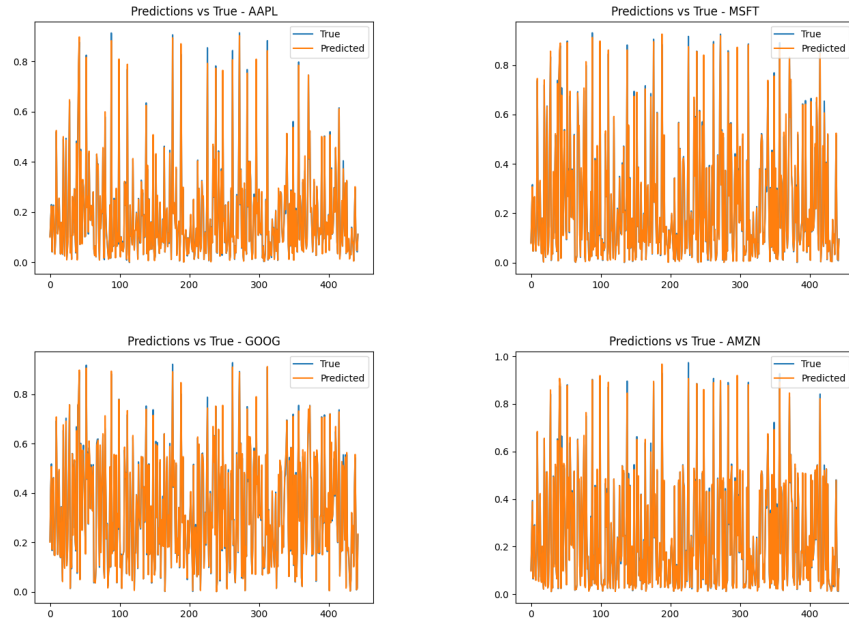


Figure 4: True vs Predicted Stocks

Figure 4 illustrates the comparison between the predicted and true stocks process for all four datasets. Overall, the GRU model is able to closely follow the general trends of the true stock prices, indicating that it successfully captures the underlying temporal patterns in the data. In particular, the predictions align with major upward and downwards movements, showing that the model effectively learn long-term dependencies.

However, the plots also reveal that the model struggles to perfectly match sharp spikes and highly volatile short-term fluctuations (same as RNN). This is expected in financial time series, as sudden market changes are often influenced by external factors that are not present in the historical price data. Despite this limitation, the GRU predictions remain relatively stable and do not exhibit large deviations from the true values.

5 Task 5: LSTM Learning

5.1 Model Structure

Next we replaced the GRU block with a Long Short-Term Memory (LSTM) block. LSTM networks are specifically designed to capture long-term dependencies in sequential data and address the vanishing gradient problem more effectively than both standard RNNs and GRUs. However, GRU generally performs better with smaller datasets/sequences.

The LSTM model consists of:

- An input sequence of 50 previous trading days.
- An LSTM recurrent layer that processes temporal dependencies using memory cells and gating mechanisms.
- A fully connected linear layer that maps the final hidden state to the predicted next-day stock price.

Unlike GRUs, LSTMs use three gates:

- Input Gate: Controls how much new information is stored in memory.
- Forget Gate: Determines what information should be discarded.
- Output Gate: Controls what information is used to compute the output

These mechanisms allow LSTMs to maintain a more precise memory of long-term patterns in stock price sequences. The same preprocessing steps, optimizer (Adam), batch size, and training setup from previous tasks were used to ensure a fair comparison. The tuned learning rate is 0.005, and the number of neurons in the hidden layer is instead 80.

5.2 Results

Name	Accuracy	Train loss	Test Loss	Time Cost
AAPL	94.55%	0.000094	0.000090	51.05 sec
MSFT	95.39%	0.000101	0.000098	45.11 sec
GOOG	96.81%	0.000154	0.000164	40.81 sec
AMZN	97.09%	0.000103	0.000103	46.59 sec

The LSTM model achieves strong predictive performance across all four stocks, with test accuracies close to or exceeding the 95% target. In particular, MSFT, GOOG and AMZN all surpass 95% accuracy, while AAPL achieves a slightly lower accuracy of 94.55%. Despite this, the overall performance remains highly competitive and comparable to the GRU model. Compared to the GRU model, the LSTM produces similar or slightly worse results for certain stocks. This

suggests that the additional memory cell structure of LSTM isn't being taken advantage of with our current data range.

The added complexity of LSTM does not always guarantee better performance. The training and test losses remain very close for all stocks, indicating minimal overfitting and strong generalization ability. Although LSTM models are generally more computationally complex than GRU models, the observed training time in this experiment was lower for LSTM (comparable to RNN). This is likely due to the relatively small dataset size, where computational overhead dominates execution time, as well as optimized backend implementations in PyTorch that can make LSTM operations highly efficient.

5.3 Learning Process Loss Change

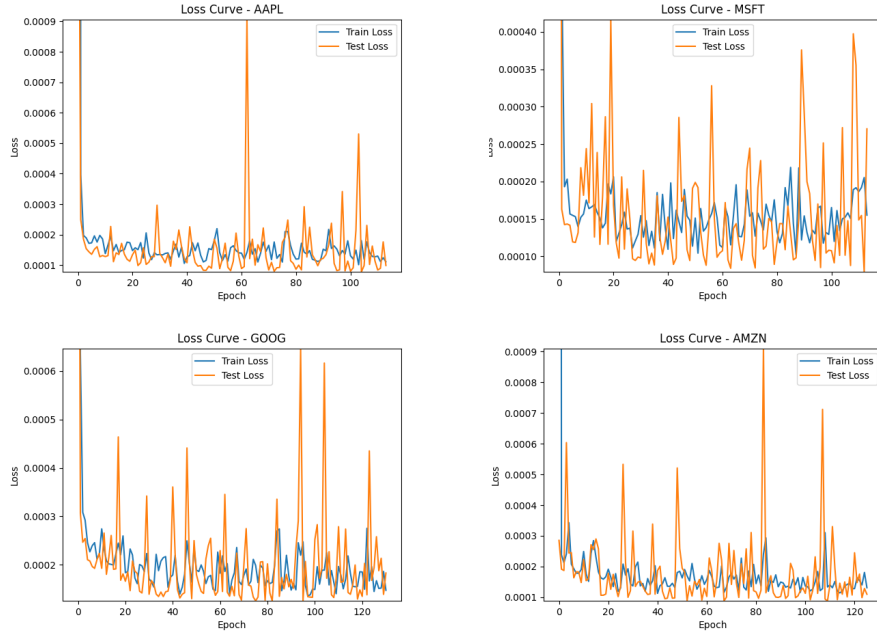


Figure 5: Training- Loss vs Epochs

Figure 5 shows the training and test loss curves for the LSTM model across all four stocks. In each case, both training and test losses decrease sharply within the first few epochs, indicating that the model quickly learns meaningful patterns from the input sequences. After this rapid drop, the loss values stabilize at low levels, demonstrating effective convergence of the model. The close alignment between the training and test loss curves across all stocks suggests that the model does not suffer from significant overfitting and generalizes well to unseen data. While there are noticeable fluctuations and occasional spikes in

the test loss (more significant than RNN and GRU) these are expected due to the inherent noise and unpredictability of financial time series data rather than instability in the model itself. Overall, these loss curves demonstrate that the selected hyperparameters and model design lead to decent training performance.

5.4 Predicted vs True Stocks

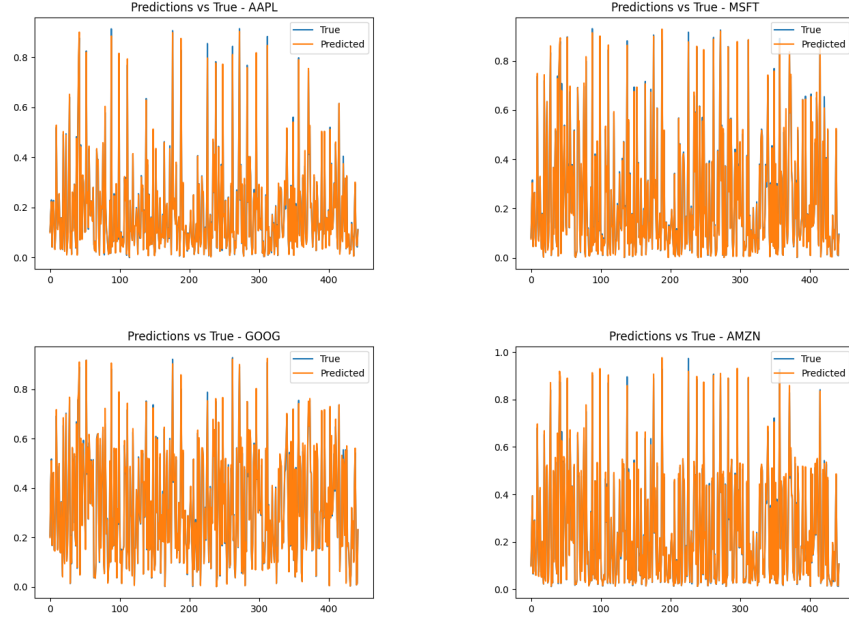


Figure 6: True vs Predicted Stocks

Figure 6 presents the comparison between the predicted and true stock prices for the LSTM model across all datasets. Overall, the LSTM model is able to capture the general distribution and range of stock price movements, with predictions remaining close to the true values in most regions. The model performs well in following broader trends and maintaining consistency in prediction magnitudes. However, similar to the GRU model, the LSTM struggles to precisely match sharp spikes and highly volatile fluctuations in the stock prices. This behavior is expected, as sudden market changes are often influenced by external factors not included in the data.

Compared to the GRU model, the LSTM produces slightly smoother and more stable predictions due to its more complex memory structure. This allows it to better retain long-term dependencies, although it may slightly smooth out extreme variations. Despite this, the predictions remain accurate overall, as reflected in the low test errors and high accuracy values. Overall, the LSTM

model effectively captures the underlying patterns in stock price sequences and provide reliable predictions, while maintaining strong generalization across all stocks.